

Measurements, Files and Spreadsheets

Engineering Computation with Python — Chapter 10

Muním Zabidi

Faculty of Electrical Engineering, Universiti Teknologi Malaysia

- 1** **From the Console to a File**
- 2** **Formatted Output with f-Strings**
- 3** **Files: Text, Binary, and Beyond**
- 4** **Spreadsheets and CSV**
- 5** **Case Study: Mesh Circuits**

1 From the Console to a File

2 Formatted Output with f-Strings

3 Files: Text, Binary, and Beyond

4 Spreadsheets and CSV

5 Case Study: Mesh Circuits

Up to now, every number in your programs was typed directly into the code. Real engineering data arrives differently.

Prompt data

Entered or displayed at the console while a script runs.
Temporary — it disappears once the program closes.

Import/export data

Stored in a file (.csv, .xlsx, .txt, ...). Permanent, shareable, and usually larger.

This chapter's purpose

Close the gap between a laboratory measurement and a Python calculation — both directions.



Consider the bench recording from earlier chapters. Loading it takes one line, and the first thing to do afterwards is always to **look at what arrived**.

```
import pandas as pd

data = pd.read_csv('bench_rc.csv')
print(data.shape)      # (101, 4)
print(data.head(3))
```

Engineering sanity checks, every time:

- **Shape.** 101 rows at 20 ms covers two seconds – matches the experiment.
- **Units.** Column names carry them: volts, milliamps, °C.
- **Magnitude.** First current 2.55 mA matches $12\text{ V} / 4.7\text{ k}\Omega$.
- **Completeness.** `data.isna().sum()` finds the logger dropout.

Requesting Input from the User

`input()` always returns a **string** — numeric responses must be converted explicitly.

```
text = input("Enter a text: ")  
number = float(input("Enter a number: "))
```

```
height = float(input('Enter height: '))  
width = float(input('Enter width: '))  
area = 0.5 * width * height  
print(f'Area = {area}')
```

Console session:

```
Enter height: 10  
Enter width: 20  
Area = 100.0
```

Three Ways to Display a Value

Direct display	print()	print() + f-string
<pre>x = 10 x</pre>	<pre>x = 10 print(x)</pre>	<pre>x = 10 print(f'x is {x}')</pre>
10	10	x is 10

The difference

Direct display and `print()` show only the value. An f-string lets the output include **explanatory text** along with it — the next section develops this fully.

1 From the Console to a File

2 **Formatted Output with f-Strings**

3 Files: Text, Binary, and Beyond

4 Spreadsheets and CSV

5 Case Study: Mesh Circuits

An f-string offers far more control than a basic `print()`: width, decimal places, and notation — all inside the braces.

```
formatted_string = f"text {variable1:.2f} more text {variable2:d}"
```

- Curly braces `{}` are placeholders for variables.
- A **format specifier** after the colon controls width and precision (e.g. `.2f` = 2 decimal places).
- Formatting affects only **display** — never the stored value.

Value Type	Code	Example	Meaning
Integer	d	{v:d}	Signed decimal integer
Float	f	{v:7.2f}	Width 7, 2 decimal places
String	s	{v:s} or {v}	Plain string output
Scientific	e/E	{v:e}	Exponential notation

```
cows = 5  
print(f'There are {cows:7.2f} cows in the pasture')
```

```
There are    5.00 cows in the pasture
```

Reading {cows:7.2f}

7 sets a minimum field width of 7 characters (digits, point, and padding). .2f sets 2 decimal places.

Consistent column widths are what make a printed table readable:

```
L = [[20, 3.3], [20000, 3300]]  
print(f'{L[0][0]:2.2f}m is equivalent to {L[1][0]:5d}mm')  
print(f'{L[0][1]:2.2f}m is equivalent to {L[1][1]:5d}mm')
```

```
20.00m is equivalent to 20000mm  
3.30m is equivalent to  3300mm
```

A 2D list is indexed `L[row][column]` – double brackets, not `L[row, column]`.

1 From the Console to a File

2 Formatted Output with f-Strings

3 **Files: Text, Binary, and Beyond**

4 Spreadsheets and CSV

5 Case Study: Mesh Circuits

Direction

- **Import** — disk → Python
- **Export** — Python → disk

Representation

- **Text** — readable (CSV, TXT, JSON)
- **Binary** — not readable in a text editor (images, audio, serialized objects)

The pattern, regardless of format

`open()` creates a file object; `read()` and `write()` move data through it. Everything that follows is a variation on this one idea.

```
# Writing to a text file
with open('example.txt', 'w') as file:
    file.write('Hello, World!')

# Reading from a text file
with open('example.txt', 'r') as file:
    content = file.read()
    print(content) # Output: Hello, World!
```

The with statement

Opens the file, hands it to file, and **closes it automatically** when the block ends — even if an error occurs inside.

Binary Data: Same Pattern, Different Mode

Adding `b` to the mode writes raw bytes — for data not meant to be read by eye, like instrument recordings.

Serializing a Python object:

```
import pickle
data = {'key': 'value'}
with open('data.pkl', 'wb') as f:
    pickle.dump(data, f)

with open('data.pkl', 'rb') as f:
    loaded = pickle.load(f)
```

Saving a NumPy array:

```
import numpy as np
arr = np.array([1, 2, 3])
np.save('array.npy', arr)

loaded = np.load('array.npy')
```

`pickle` serializes any Python object; `numpy.save/load` is the NumPy-specific equivalent.

- **Text file** (open, read, write) — small, human-readable notes or logs.
- **pickle** — saving an arbitrary Python object (a dictionary, a trained model) exactly as it was.
- **NumPy .npy** — saving a large array efficiently, without converting to text.
- **CSV / spreadsheet** — tabular measurement data meant to be shared, inspected, or opened in Excel.

This chapter's focus

Spreadsheets and CSV are the formats engineers meet daily — the rest of this chapter covers them in depth.

1 From the Console to a File

2 Formatted Output with f-Strings

3 Files: Text, Binary, and Beyond

4 **Spreadsheets and CSV**

5 Case Study: Mesh Circuits

```
# Reading data
data = pd.read_excel(filename, sheet_name='Sheet1',
                    usecols='B:Z', header=None)

# Selecting rows/columns
selected_data = data.iloc[1, 1:]

# Writing data back
output_data.to_excel(filename, sheet_name='Sheet1',
                    startcol=1, startrow=2,
                    index=False, header=False)
```

startcol/startrow target where writing begins; index=False skips saving the DataFrame's row index into the sheet.

Worked Example: Celsius to Fahrenheit

A room temperature log sits in `tempData.xlsx`. Convert every reading to Fahrenheit and append it to the same file.

```
temp_c = pd.read_excel(file_path, sheet_name='Sheet1',
                      header=None).iloc[1, 1:]
temp_f = temp_c * 9/5 + 32

with pd.ExcelWriter(file_path, engine='openpyxl', mode='a',
                   if_sheet_exists='overlay') as writer:
    output_data = pd.DataFrame([temp_f])
    output_data.to_excel(writer, sheet_name='Sheet1',
                        startcol=1, startrow=2,
                        index=False, header=False)
```

`mode='a'` with `if_sheet_exists= 'overlay'` adds data **without erasing** what is already there.

Excel vs. CSV: Same Idea, Fewer Moving Parts

Feature	Excel	CSV
Library	openpyxl required	Only pandas
Reading	<code>pd.read_excel(...)</code>	<code>pd.read_csv(...)</code>
Writing	<code>pd.ExcelWriter(...)</code>	<code>.to_csv(...)</code>
Sheets	sheet_name	None
Write mode	Overlay existing cells	Append a new row
Cell targeting	startrow, startcol	Not supported

CSV is the default for measured data

Simpler, universally readable, and exactly what a data logger or lab instrument usually exports.

```
data = pd.read_csv(file_path, header=None)
temp_c = data.iloc[1, 1:]
temp_f = temp_c * 9/5 + 32

# CSV has no cells to target -- just append a new row
output_data = pd.DataFrame([temp_f])
output_data.to_csv(file_path, mode='a',
                    index=False, header=False)
```

No sheet name, no startrow/startcol – CSV has no concept of cell targeting, so the converted row is simply appended at the end.

1 From the Console to a File

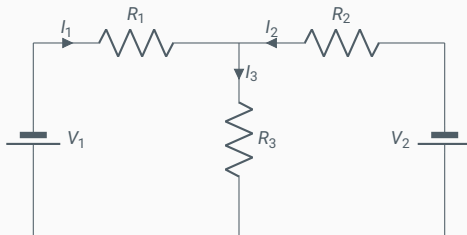
2 Formatted Output with f-Strings

3 Files: Text, Binary, and Beyond

4 Spreadsheets and CSV

5 **Case Study: Mesh Circuits**

Revisit the mesh-circuit analysis from Chapter 9, now driven entirely by spreadsheet data: solve I_1 and I_2 for **ten** resistor/voltage configurations stored in `circuit_analysis.xlsx`.



The script sends R (resistances) and V (voltages) from the spreadsheet into a helper module, `mesh_solver.py`, for each of the ten rows.

Columns B–D hold R_1, R_2, R_3 ; columns E–F hold V_1, V_2 ; columns G–H are reserved for the **computed** I_1, I_2 .

	R_1	R_2	R_3	V_1	V_2
Set 1	11	2	3	10	−40
Set 2	5	20	10	10	−20
Set 3	16	16	16	20	20
...	...	...	...	...	...
Set 10	100	25	100	−20	10

Ten rows in, ten pairs of currents out — by hand, this would mean solving the same 2×2 system ten times.

Step 1: Separating Resistances from Indicators

The R array packs resistor values **and** which loop each belongs to into one array:

$$R = \begin{bmatrix} R_1 & R_2 & R_3 \\ 1 & 0 & 1 \\ 0 & 1 & 1 \end{bmatrix}$$

```
def separate_ir(R):  
    r = R[0, :]  
    i = R[1:, :].astype(bool)  
    total_loops = R.shape[0] - 1  
    return i, r, total_loops
```

resistor values
which loop each belongs to

```
def mesh_solver(V, R):  
    i, r, M = separate_ir(R)  
    Rmat = np.zeros((M, M))  
    for m in range(M):  
        for n in range(M):  
            ind_i = np.logical_and(i[m, :], i[n, :])  
            if n != m:  
                Rmat[m, n] = -np.sum(r[ind_i])  
            else:  
                Rmat[m, n] = np.sum(r[ind_i])  
    I = np.linalg.solve(Rmat, V)  
    return I
```

The same operation as Chapter 9's $Ax = b$ — now driven entirely by data read from a file.

```
df = pd.read_excel(file_path, sheet_name='Sheet1')
data = df.to_numpy()
I1_list, I2_list = [], []

for row in data:
    R1, R2, R3 = row[1:4]
    V1, V2 = row[4:6]
    R = np.array([[R1, R2, R3], [1, 0, 1], [0, 1, 1]])
    V = np.array([V1, V2])
    I = mesh_solver(V, R)
    I1_list.append(I[0]); I2_list.append(I[1])
```

One loop, ten configurations solved – the pattern scales to a hundred rows exactly as easily.

```
I_values_df = pd.DataFrame({'I1': I1_list, 'I2': I2_list})  
  
with pd.ExcelWriter(file_path, mode='a',  
                    if_sheet_exists='overlay') as writer:  
    I_values_df.to_excel(writer, startcol=6, startrow=1,  
                        index=False, header=False)
```

The full cycle, completed

Data moved from spreadsheet → Python → spreadsheet, with the original columns untouched and I_1 , I_2 filled in alongside them.

This case study is not really about mesh circuits — it is a template for almost any engineering computation on tabular data:

1. Read the spreadsheet into a DataFrame.
2. Loop over rows, extracting the inputs each row needs.
3. Call a tested function (`mesh_solver`) that does the actual engineering calculation.
4. Collect the results and write them back.

The same shape reappears

Swap `mesh_solver` for a stress calculation, a calibration curve, or a filter design, and the surrounding read → loop → write structure barely changes.

This chapter closed the gap between a real measurement and a Python calculation — both directions, prompt and file alike.

- `input()` always returns a string; convert explicitly before computing with it.
- f-strings control **display**, never the stored value.
- CSV is the default interchange format for measured data; Excel adds sheets and cell targeting.
- The mesh-circuit case study is the template: read, loop, compute, write back.

Next: Errors, Missing Data and Outliers